```
#include <iostream>
#include <cuda_runtime.h>
using namespace std;

__global__ void vectorAdd(int *A, int *B, int *C, int n) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < n) {
        C[i] = A[i] + B[i];
    }
}

int main() {
    int n;
    cout << "Enter size of vectors: ";
    cin >> n;

    int *h_A = new int[n];
    int *h_B = new int[n];
    int *h_C = new int[n];

    cout << "Enter elements of A:\n";
    for (int i = 0; i < n; i++)
        cin >> h_A[i];

    cout << "Enter elements of B:\n";
    for (int i = 0; i < n; i++)
        cin >> h_B[i];

    int *d_A, *d_B, *d_C;

    cudaMalloc((void**)&d_A, n * sizeof(int));
    cudaMalloc((void**)&d_B, n * sizeof(int));
    cudaMalloc((void**)&d_C, n * sizeof(int));

    cudaMemcpy(d_A, h_A, n * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, n * sizeof(int), cudaMemcpyHostToDevice);

    int blockSize = 256;
    int gridSize = (n + blockSize - 1) / blockSize;

    vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, n);
    cudaDeviceSynchronize();

    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        cout << "CUDA Error: " << cudaGetErrorString(err) << endl;
        return 1;
    }

    cudaMemcpy(h_C, d_C, n * sizeof(int), cudaMemcpyDeviceToHost);

    cout << "\nResultant vector:\n";
    for (int i = 0; i < n; i++)
        cout << h_C[i] << " ";
```

```
    cout << endl;

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    delete[] h_A;
    delete[] h_B;
    delete[] h_C;

    return 0;
}
```